

Design a Local Delivery Service

📖 Scaling Reads

By Stefan Mai · Published Jan 8, 2024 · 🟢 easy



Try This Problem Yourself

Practice with guided hints and real-time feedback



Start Practice

Understanding the Problem

🔍 What is Gopuff?

Gopuff delivers goods typically found in a convenience store via rapid delivery and 500+ micro distribution centers (DCs).

Functional Requirements

We'll start by asking our interviewer about the extent of the functionality we need to build. Our goal for this section is to produce a concise set of functionalities that the system needs to support. In this case, we need to be able to query availability of items in our Distribution Centers (DCs) and place orders.

Core Requirements

1. Customers should be able to query availability of items, deliverable in 1 hour, by location (i.e. the effective availability is the union of all inventory nearby DCs).
2. Customers should be able to order multiple items at the same time.

Below the line (out of scope)

- Handling payments/purchases.
- Handling driver routing and deliveries.
- Search functionality and catalog APIs. (The system is strictly concerned with availability and ordering).
- Cancellations and returns.



For this problem, the emphasis is on aggregating availability of items across local distribution centers and allowing users to place orders without double booking. In other problems you may be more concerned with the product catalog, search functionality, etc.

Non-Functional Requirements

Core Requirements

1. Availability requests should be fast (<100ms) to support use-cases like search.
2. Ordering should be strongly consistent: two customers should not be able to purchase the same physical product.
3. System should be able to support 10k DCs and 100k items in the catalog across DCs.

4. Order volume will be $O(10m \text{ orders/day})$

Below the line (out of scope)

- Privacy and security.
- Disaster recovery.

Here's how these might be shorthanded in an interview. Note that out-of-scope requirements usually stem from questions like "do we need to handle privacy?". Interviewers are usually comfortable with you making assertions "I'm going to leave privacy out of scope for the start" and will correct you if needed.

Functional

- * Query availability by location (inside 1 hour)
- * Order items

Out of Scope

- * Payments/purchases
- * Driver routing
- * Search
- * Cancellations and returns

Non-functional

- * Can connect to any DC within 1 hour
- * Fast (100ms) for availability
- * Ordering is strongly consistent
- * 10k DCs, 100k items in catalog
- * Order volume $O(1m)$

Requirements



Set Up

Planning the Approach

Before we go forward with designing the system, we'll think briefly through the approach we'll take.

For this problem, our requirements are fairly straightforward, so we'll take our default approach. We'll start by designing a system which simply supports our functional requirements without much concern for scale or our non-functional requirements. Then, in our deep-dives, we'll bring back those concerns one by one.

To do this we'll start by enumerating the "nouns" of our system, build out an API, and then start drawing our boxes.

Defining the Core Entities

Core entities are the high level "nouns" for the system. We're not (yet) designing a full data model - for that, we'll need to have a better idea of how the system fits together overall. But by figuring out the entities we'll have the right blocks to build the rest of the system. Getting the right entities is particularly important for designing a good REST API as resources are the central anchor point for a REST API - which are very tightly related to our core entities.

Core Entities

Item

Inventory

Distribution Center

Distribution Center

Order

Order Item



Core Entities

One important thing to note for this problem is the distinction between **Item** and **Inventory**. You might think of this like the difference between a Class and an Instance in object-oriented programming. While our API consumers are strictly concerned with items that they might see in a catalog (e.g. Cheetos), we need to keep track of where the **physical** items are actually located. Our **Inventory** entity is a physical item at a specific location.

The remaining entities should be more obvious: we need **DistributionCenter** to keep up with where our inventory is physically located and an **Order** entity to keep track of the items being ordered.

- **Inventory**: A physical instance of an item, located at a DC. We'll sum up Inventory to determine the quantity available to a specific user for a specific **Item**.
- **Item**: A type of item, e.g. Cheetos. These are what our customers will actually care about.
- **DistributionCenter**: A physical location where items are stored. We'll use these to determine which items are available to a user. **Inventory** are stored in DCs.
- **Order**: A collection of **Inventory** which have been ordered by a user (and shipping/billing information).



Particularly in product design, outlining the **identity** of the entities in your system is important. By starting with the most concrete physical or business entities (e.g. items, users, etc.) and working your way up to more abstract entities (e.g. orders, carts, etc.) you can ensure that you don't miss any important entities.

Defining the API

Next, we can start with the APIs we need to satisfy, which should track closely to our functional requirements. There are a lot of extraneous details we could shove into these APIs, but we're going to start simple to avoid burning unnecessary time in the interview (a common mistake in practice!).

To meet our requirements we only need two APIs: the first API allows us to get availability of items given a location (and maybe a keyword), and the second API allows us to place an order. We'll include pagination in our availability API to avoid overwhelming the client with more data than it needs.

```
GET /v1/availability?lat=LAT&long=LONG&keyword={}&page_size={}&page_num={} ->
{
  items: {
    name: NAME
    quantity: QTY
  }[]
}
```

```
POST /v1/order
{
  lat: LAT,
  long: LONG
}
```

items: ITEM1,ITEM2,ITEM3...

...
} -> Order | Failure



API



Note here that we're passing our location to both APIs: before the order can be processed by the backend we'll need to confirm the inventory is available close enough to the user's location to deliver within 1 hour.

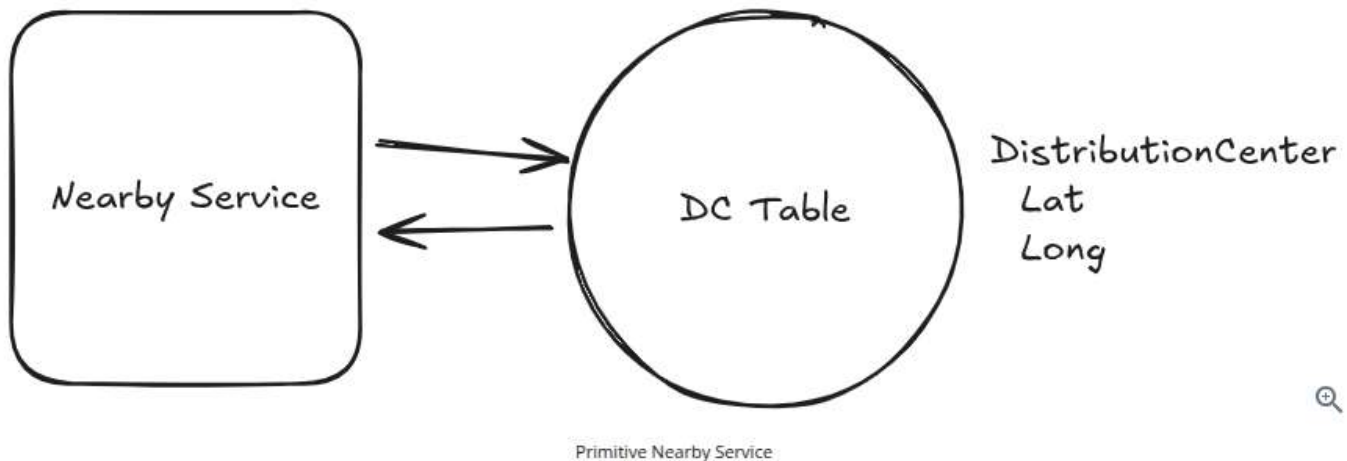
High-Level Design

1) Customers should be able to query availability of items

Let's move to the first requirement: customers should be able to query availability of items by location.

To do this, we have two steps. First, we need to find the DCs that are close enough to deliver in 1 hour. All of our inventory *lives* in a DC, so we can shortcut having to check every item in our system. Next, once we have the list of serviceable DCs, we can check the inventory of them and return the union to the user. We need each step to be reasonably fast since our end-to-end latency should be less than 100ms.

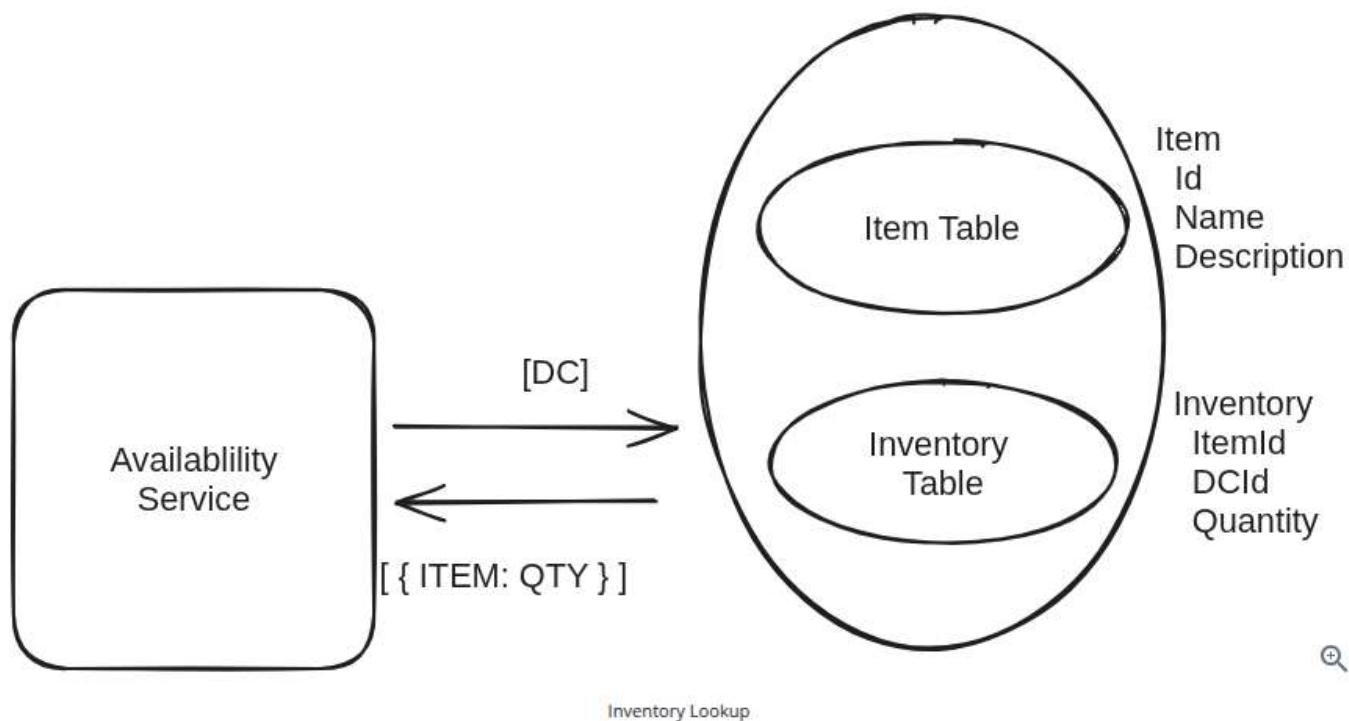
To find nearby DCs, we can build a simple internal API which takes a LAT and LONG and returns a list of DCs within 1 hour. Let's assume we have a table of DCs with their lat/long. Crudely, we can measure the distance to the user's input with some simple math. A very basic version might use Euclidean distance while a more sophisticated example might use the Haversine formula to take into account the curvature of the Earth. Taking a simple threshold on this query would give us DCs within X distance as the crow flies. This isn't quite satisfying our functional requirement, but we'll come back to this in our deep dive.



Next we need to check the inventory of the DCs we found. We can do this by querying our Inventory table and Items table. Let's assume we're using a Postgres database for these tables which will allow us to join this inventory table with our Item table to get the item name and description before returning with the quantity.



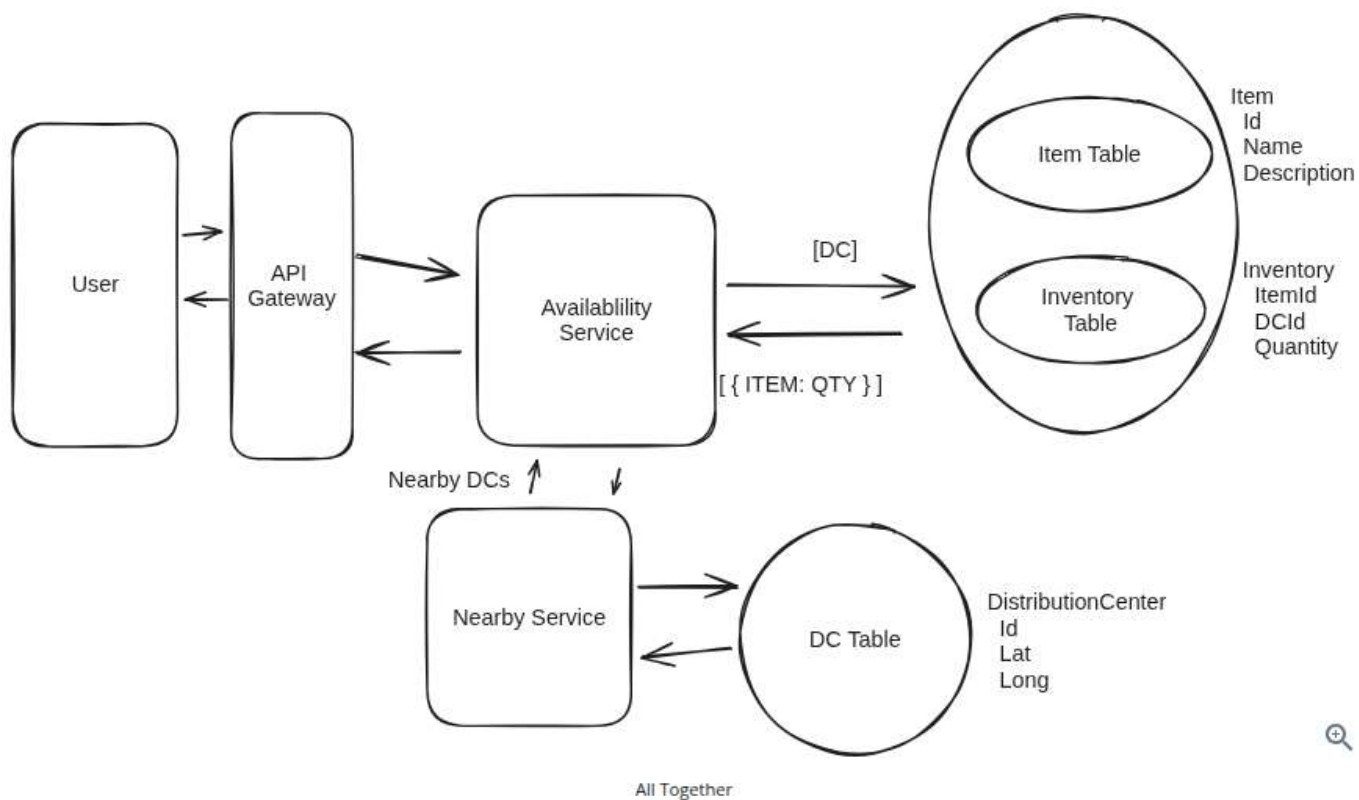
In many e-commerce systems the "Catalog" is stored separately from the inventory because of the different consumers and workloads. We'll store them in the same database here to make our job easier and to adhere to our requirements, but we might note to our interviewer that we'd ideally separate these, add a search index (like Elasticsearch) to allow searching the catalog, etc.



We now have:

1. **Availability Service** handles requests from our users for availability given a specific location.
2. **Nearby Service** syncs with the database of nearby DCs and uses an external "Travel Time Service" to calculate travel times from DCs (potentially including traffic).
3. **Inventory Table** a replicated SQL database table which returns the inventory available for each item and DC.

Putting it all together we have:



When a user makes a request to get availability for items A, B, and C from latitude X and longitude Y, here's what

happens:

1. We make a request to the **Availability Service** with the user's location X and Y and any relevant filters.
2. The availability service fires a request to the **Nearby Service** with the user's location X and Y.
3. The nearby service returns us a list of DCs that can deliver to our location.
4. With the DCs available, the availability service query our database with those DC IDs.
5. We sum up the results and return them to our client.

Great! Our availability requirement is (mostly) satisfied.

2) Customers should be able to order items.

The last thing we need to complete our requirements is for us to enable placing orders. For this, we require strong consistency to make sure two users aren't ordering the same item. To do this we need to check inventory, record the order, and update the inventory together atomically.

While latency is not a big concern here (our users will tolerate more latency here than they will on the reads), we definitely want to make sure we're not promising the same inventory to two users. How do we do this?

This idea of ensuring we're not "double booking" is a common one across system design problems. To ensure we don't allow two users to order the same inventory, we need some form of locking. The idea being that we need to "lock" the inventory while we're checking it and recording the order in a such a way that only one user can hold the lock at a time.

Good Solution: Two data stores with a distributed lock

Approach

We can have separate databases for orders and inventory. When an order to placed we'll lock the relevant inventory records, create the order record, decrement the inventory, and release the lock.

This is a good solution because it allows us to use the best data store for each use case. For example, we can use a key-value store for inventory and a relational database for orders. But ...

Challenges

It has some nasty failure modes!

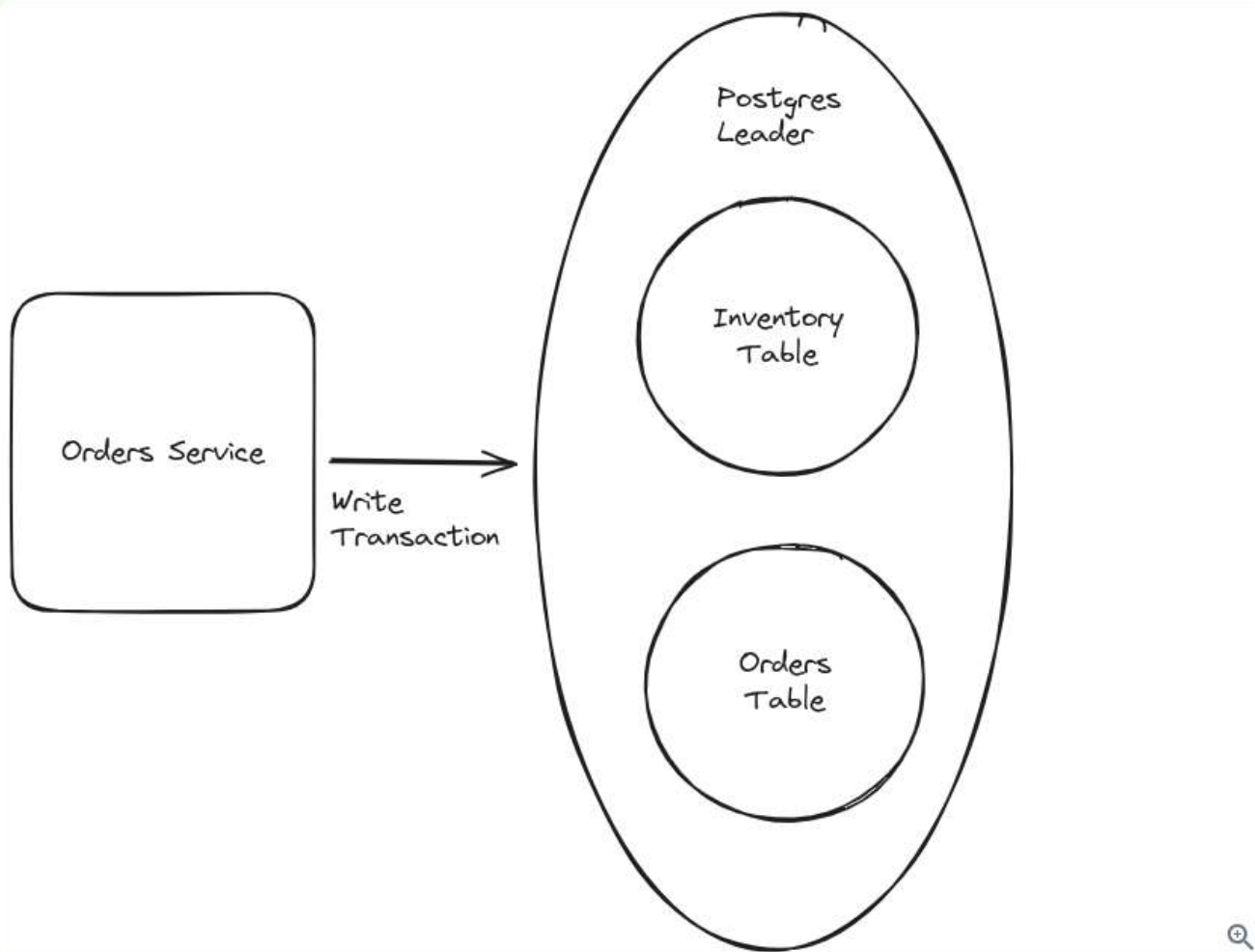
- What if our service crashes after we created the order but before we decremented the inventory? A subsequent user might order the inventory we had promised to the first user. We'll need to sweep for these failures and reverse them.
- What if two orders have overlapping inventory requirements? We might deadlock if both User1 and User2 are trying to buy A and B, but User1 has the lock for A and User2 has the lock for B - neither can proceed.

If you go this route you'll need to be able to address each of these failure modes.

Great Solution: Singular Postgres transaction

Approach

By putting both orders and inventory in the same database, we can take advantage of the ACID properties of our Postgres database. Using a singular transaction with isolation level `SERIALIZABLE` we can ensure that the entire transaction is atomic. This means that if two users try to order the same item at the same time, one of them will be rejected. This is because the transaction will fail to commit if the inventory is not available.



Singular Postgres Transaction

Challenges

While consolidating our data down to a single database has a lot of benefits, it's not without drawbacks. We're partly coupling the scaling of inventory and orders and we can't take advantage of the best data store for each use case.



When atomicity of transactions is a requirement, it's helpful to have your data colocated in an ACID data store. While it's possible to manage transactions across multiple data stores (and you may want this for other reasons), the additional complexity and overhead to support it is not what we want to focus on during this interview.

By choosing the "great" option and leaning in to our existing Postgres database we can keep our system simple and still meet our requirements. For an order, the process looks like this:

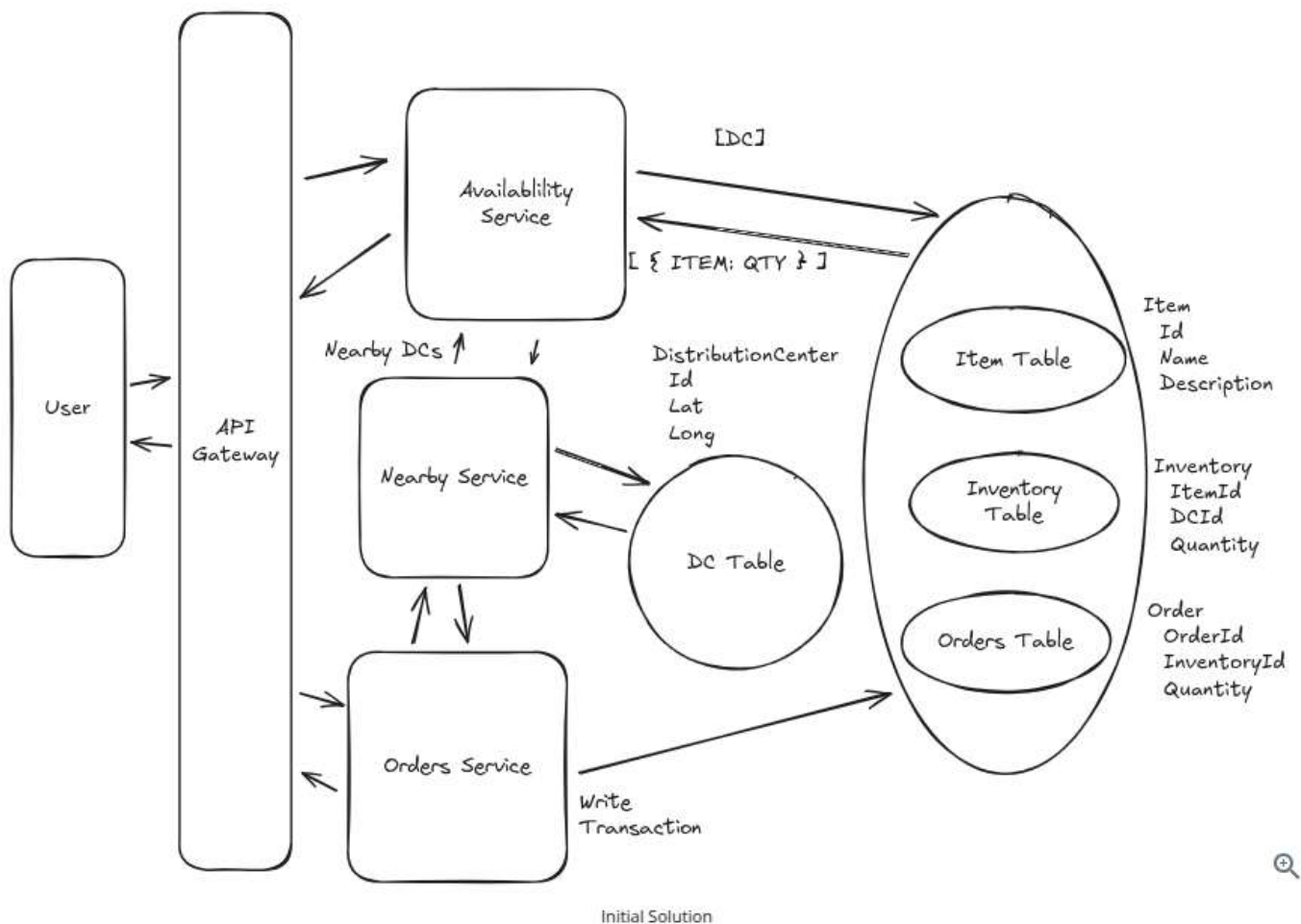
1. The user makes a request to the **Orders Service** to place an order for items A, B, and C.
2. The **Orders Service** makes creates a singular transaction which we submit to our Postgres leader. This

2. The **Orders Service** makes a single transaction when we submit to our Postgres leader. This transaction:
 - a. Checks the inventory for items A, B, and C > 0.
 - b. If any of the items are out of stock, the transaction fails.
 - c. If all items are in stock, the transaction records the order and updates the status for inventory items A, B, and C to "ordered".
 - d. A new row is created in the Orders table (and OrderItems table) recording the order for A, B, and C.
 - e. The transaction is committed.
3. If the transaction succeeds, we return the order to the user.

There are some downsides to this setup. In particular, if any of the items become unavailable in the users order the entire order fails. We'll want to return a more meaningful error message to the user in this case, but this is preferable to succeeding in an order that might not make sense (e.g. a device and its battery).

Putting it all Together

With the two approaches listed, we can now pull everything together for a solution to our problem:



We have three services, one for Availability requests, one for Orders, and a shared service for Nearby DCs. Both our Availability and Orders service use the Nearby service to look up DCs that are close enough to the user. We have a singular Postgres database for inventory and orders, partitioned by region. Our Availability service reads via read replicas, our Orders service writes to the leader using atomic transactions to avoid double writes. A great foundation!

Deep Dives

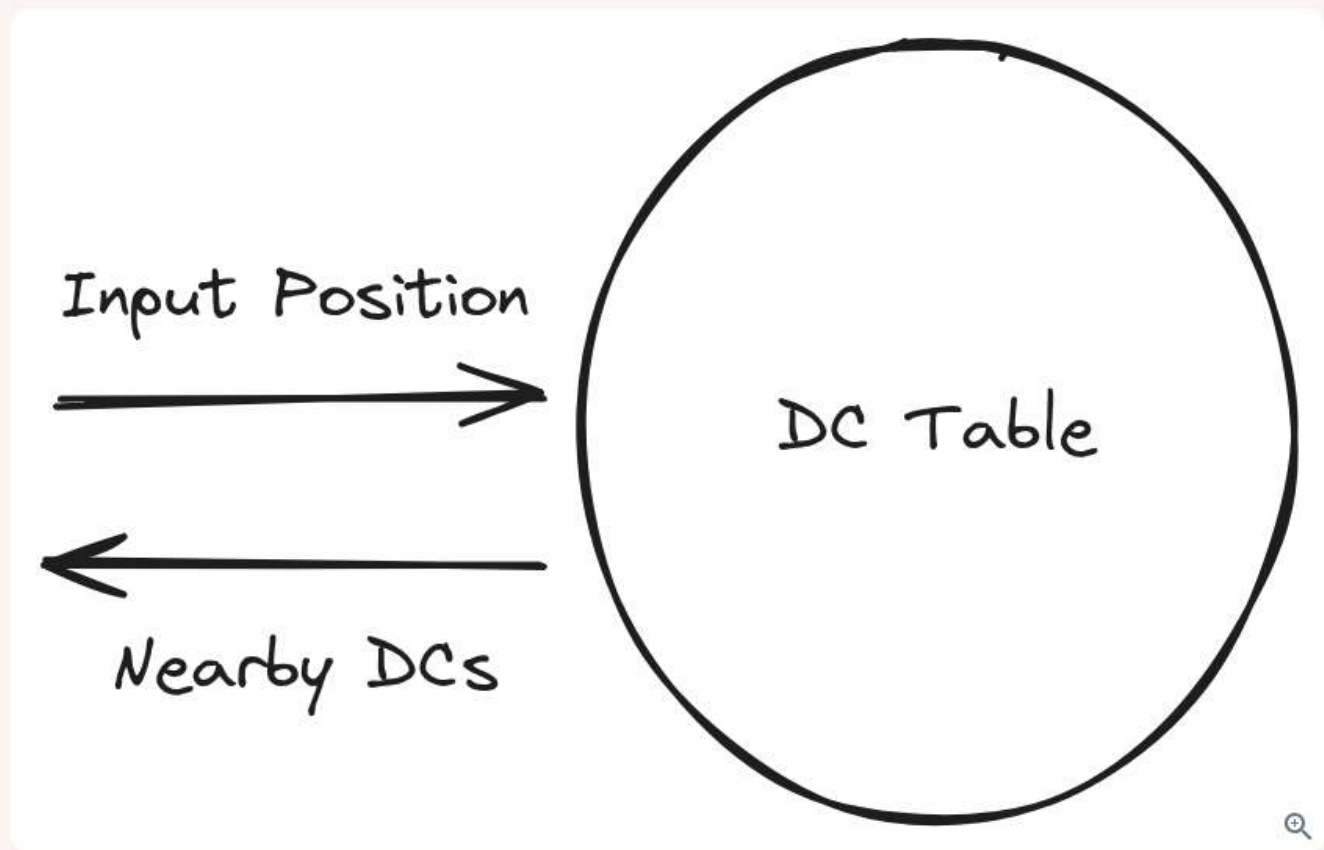
1) Make availability lookups incorporate traffic and drive time

So far our system is only determining nearby DCs based on a simple distance calculation. But if our DC is over a river, or a border, it may be close in miles but not close in drive time. Further, traffic might influence the travel times. Since our functional requirements mandate 1 hour of drive time, we'll need something more sophisticated. What can we do?

Bad Solution: Simple SQL Distance ^

Approach

We can put all the lat/long of our DC into a table, then measure the distance to the user's input with some simple math. A very basic version might use Euclidean distance while a more sophisticated example might use the Haversine formula to take into account the curvature of the Earth. Taking a simple threshold on this query would give us DCs within X as the crow flies.



Simple SQL Distance

Challenges

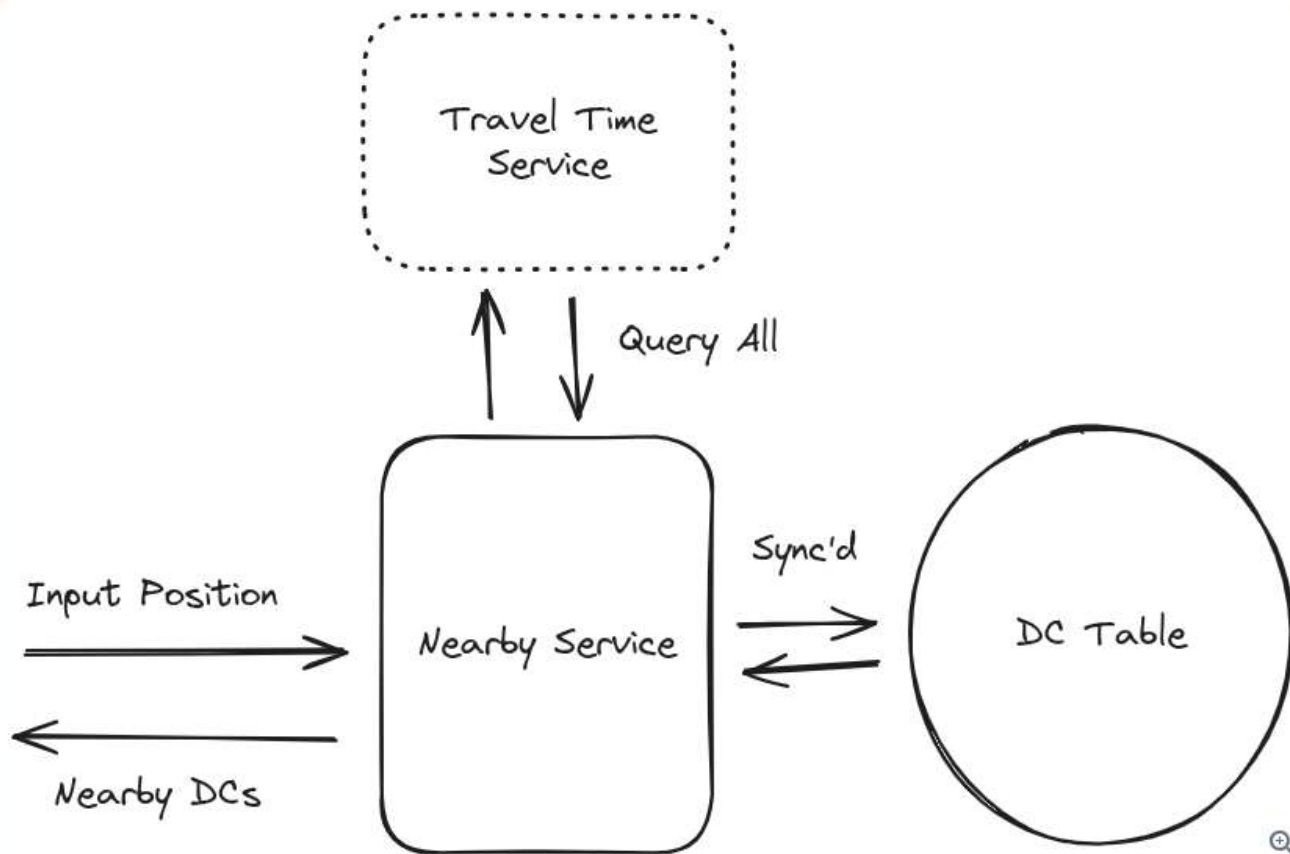
- This is a very simple approach which doesn't take into account traffic, road conditions, etc. It also doesn't take into account the fact that we might have multiple DCs in the same city.

Bad Solution: Use a Travel Time Estimation Service Against all DCs ^

Approach

Since our DCs are rarely going to change (they're buildings!), we can sync periodically (like every 5 minutes) from

a DC table to memory of our service. We can use a travel time estimation service to find the travel times from our input location to each of our DCs by iterating over all of them.



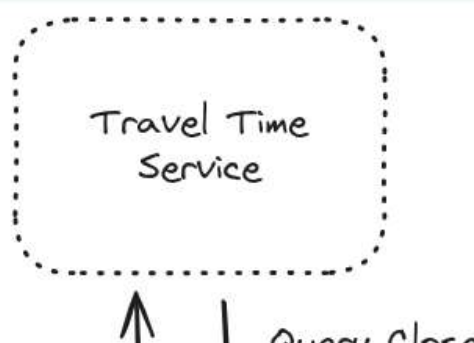
Travel Time Estimation Service

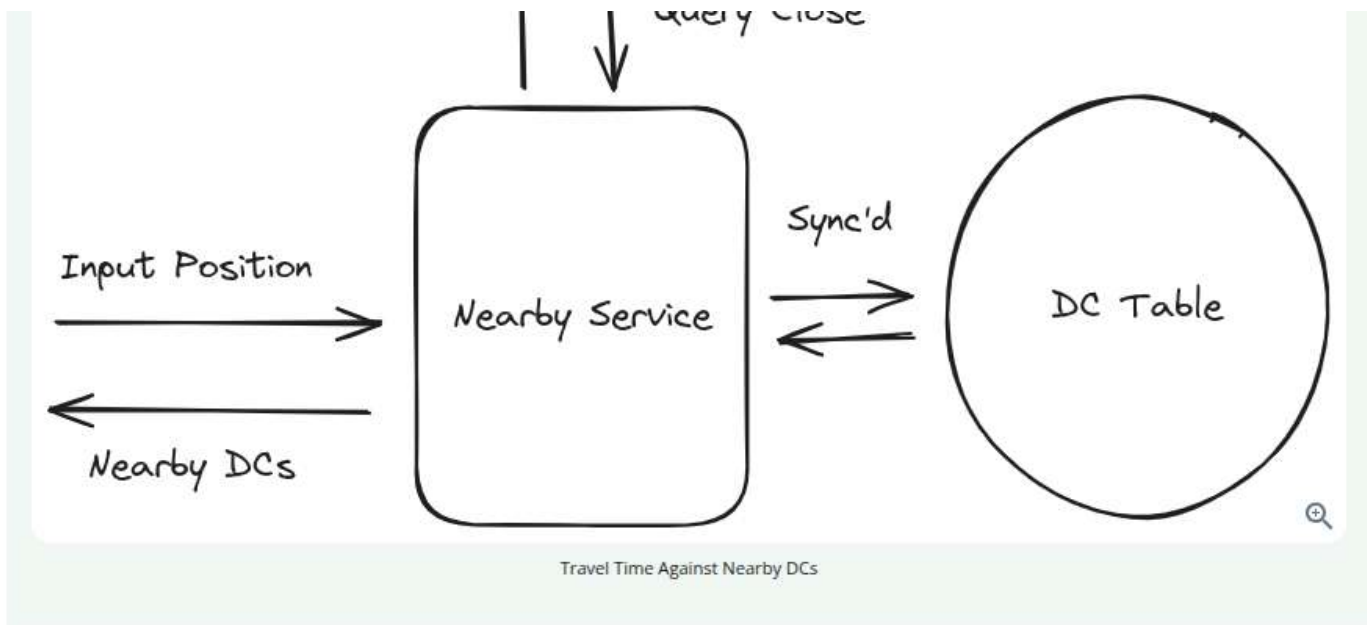
Challenges

We're making far too many queries to the travel time estimation service. Most of the DCs we're querying aren't close enough to ever plausibly be delivered in 1 hour.

Great Solution: Use a Travel Time Estimation Service Against Nearby DCs

We build on the previous solution to sync periodically (like every 5 minutes) from a DC table to memory of our service. When an input comes in, we can prune down the "candidate" DCs by taking a fixed radius (say 60 miles, the most optimistic distance we could drive over in 1 hour) and limiting ourselves to only evaluating those. We'll take these restricted candidates and then pass to the external travel time service to create our final estimate.





2) Make availability lookups fast and scalable

Currently, once we have nearby DCs we use those DCs to look up availability directly from our database. This introduces a lot of load onto our database. Let's briefly detour into some estimates to figure out how much throughput we might expect.



Using quantitative estimation where you've spotted a potential bottleneck can significantly improve your interview performance. It gives you and your interviewer a common set of data from which to weigh tradeoffs and it shows you're able to make reasonable assumptions about the system.

To figure out how many queries for availability we might have, we'll back in from our orders/day requirement which we set at 10m orders per day. All-in, we might estimate that each user will look at 10 pages across search, the homepage, etc. before purchasing 1 item. In addition, maybe only 5% of these users will end up buying where the rest are just shopping.

Queries: $10M \text{ orders/day} / (100k \text{ seconds/day}) * 10 / 0.05 = 20k \text{ queries/second}$

This is pretty sizeable number of queries per second. We clearly need to think about how we can scale this. What do we do?

Pattern: Scaling Reads

Local delivery services like Gopuff demonstrate classic **scaling reads** patterns where inventory queries vastly outnumber actual purchases. With 20k queries/second for availability checks but only occasional inventory updates, aggressive caching with short TTLs becomes critical.

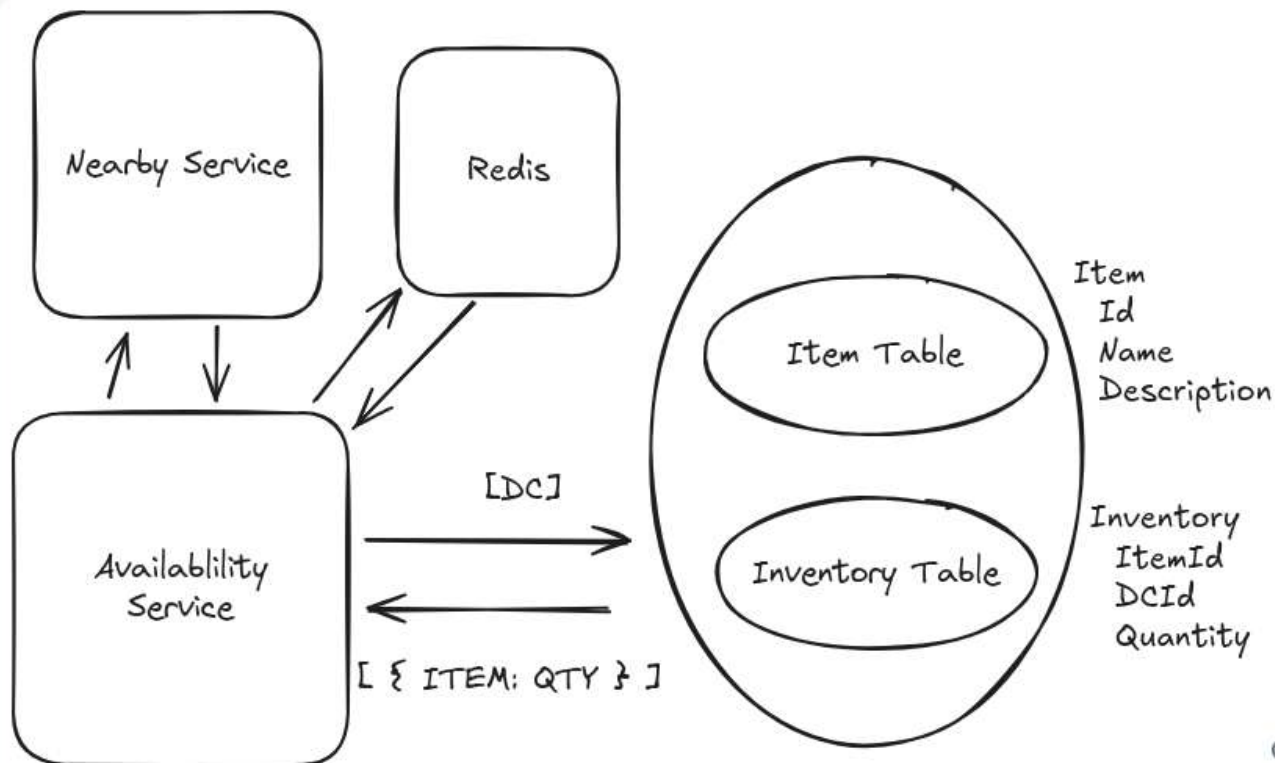
[Learn This Pattern](#)

Great Solution: Query Inventory Through Cache



Approach

We can add a Redis instance to our setup. Our availability service can query the cache for a given set of inputs and, if the cache hits, return that result. If the cache misses we'll do a lookup on the underlying database and then write the results into the cache. Setting a low TTL (e.g. 1 minute) ensures that these results are fresh.



Inventory Through Cache

Challenges

We need to ensure that the cache is always up to date with the inventory table. Our Order Service will need to expire affected cache entries when it writes to the inventory table.

Great Solution: Postgres Read Replicas and Partitioning

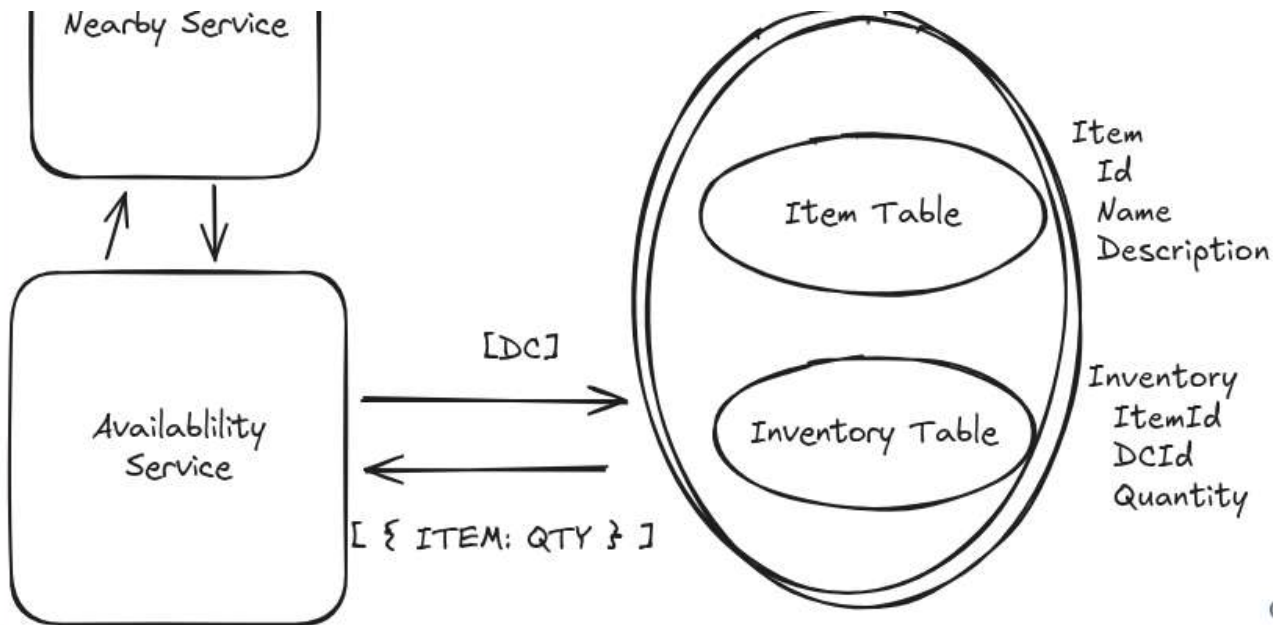
Approach

Since we only ever read inventory from a nearby collection of DC's, we can group them together with a region ID using the first 3 digits of their zipcode. Then we can partition our inventory based on this region IDs. This means all queries will go to mostly 1 or 2 partitions rather than the entire inventory dataset.

We can also use read replicas for availability since we can tolerate a small amount of inconsistency. Our orders need to be strongly consistent, so those transactions need to be sent to the Postgres leader, but our availability queries can go to our read replicas.



Read Replicas +
Sharding by REGION_ID

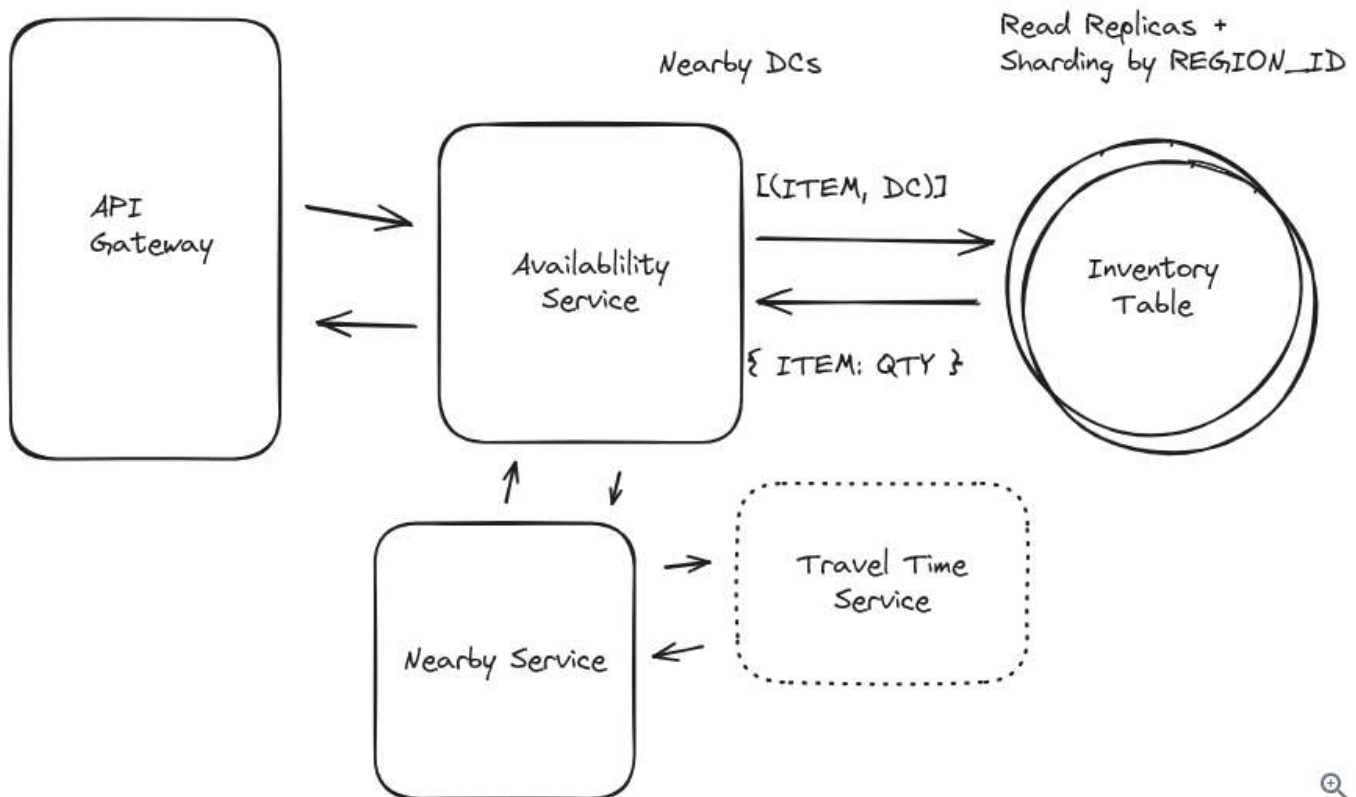


Postgres Read Replicas and Partitioning

Challenges

We'll need to manage the sizing of our replicas to balance with traffic. We'll also need to manage the partitioning of our data to ensure that we're not overloading any one replica.

By choosing the "great" solutions for both challenges we net out with a solution that looks something like this:



Final Nearby Service

What is Expected at Each Level?

Your interviewer may even have you go deeper on specific sections, or ask follow-up questions. What might you expect in an actual assessment?

Mid-Level

Breadth vs. Depth: A mid-level candidate will be mostly focused on breadth. As an approximation, you'll show 80% breadth and 20% depth in your knowledge. You should be able to craft a high-level design that meets the functional requirements you've defined, but the optimality of your solution will be icing on top rather than the focus.

Probing the Basics: Your interviewer spend some time probing the basics to confirm that you know what each component in your system does. For example, if you use DynamoDB, expect to be asked about the indexes available to you. Your interviewer will not be taking anything for granted with respect to your knowledge.

Mixture of Driving and Taking the Backseat: You should drive the early stages of the interview in particular, but your interviewer won't necessarily expect that you are able to proactively recognize problems in your design with high precision. Because of this, it's reasonable that they take over and drive the later stages of the interview while probing your design.

The Bar for GoPuff: For this question, interviewers expect a mid-level candidate to have clearly defined the API endpoints and data model, and created both routes: availability and orders. In instances where the candidate uses a "Bad" solution, the interviewer will expect a good discussion but not that the candidate immediately jumps to a great (or sometimes even good) solution.

Senior

Depth of Expertise: As a senior candidate, your interviewer expects a shift towards more in-depth knowledge — about 60% breadth and 40% depth. This means you should be able to go into technical details in areas where you have hands-on experience.

Advanced System Design: You should be familiar with advanced system design principles. Certain aspects of this problem should jump out to experienced engineers (read volume, trivial partitioning) and your interviewer will be expecting you to have reasonable solutions.

Articulating Architectural Decisions: Your interviewer will want you to clearly articulate the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability. You should be able to justify your decisions and explain the trade-offs involved in your design choices.

Problem-Solving and Proactivity: You should demonstrate strong problem-solving skills and a proactive approach. This includes anticipating potential challenges in your designs and suggesting improvements. You need to be adept at identifying and addressing bottlenecks, optimizing performance, and ensuring system reliability.

The Bar for GoPuff: For this question, a senior candidate is expected to speed through the initial high level design so we can spend time discussing, in detail, how to optimize the critical paths. Senior candidates would be expected to have optimized solutions for both the atomic transactions of the orders service as well as the scaling of the availability service.

Staff+

Emphasis on Depth: As a staff+ candidate, the expectation is a deep dive into the nuances of system design — the interviewer is looking for about 40% breadth and 60% depth in your understanding. This level is all about demonstrating that "been there, done that" expertise. You should know which technologies to use, not just in theory but in practice, and be able to draw from your past experiences to explain how they'd be applied to solve specific problems effectively. Your interviewer knows you know the small stuff (REST API, data normalization, etc) so you can

problems effectively, real-world scenarios, and you must be able to explain your solution in a clear, concise manner, as if you can breeze through that at a high level so we have time to get into what is interesting.

High Degree of Proactivity: At this level, your interviewer expects an exceptional degree of proactivity. You should be able to identify and solve issues independently, demonstrating a strong ability to recognize and address the core challenges in system design. This involves not just responding to problems as they arise but anticipating them and implementing preemptive solutions.

Practical Application of Technology: You should be well-versed in the practical application of various technologies. Your experience should guide the conversation, showing a clear understanding of how different tools and systems can be configured in real-world scenarios to meet specific requirements.

Complex Problem-Solving and Decision-Making: Your problem-solving skills should be top-notch. This means not only being able to tackle complex technical challenges but also making informed decisions that consider various factors such as scalability, performance, reliability, and maintenance.

Advanced System Design and Scalability: Your approach to system design should be advanced, focusing on scalability and reliability, especially under high load conditions. This includes a thorough understanding of distributed systems, load balancing, caching strategies, and other advanced concepts necessary for building robust, scalable systems.

The Bar for GoPuff: For a staff+ candidate, expectations are set high regarding depth and quality of solutions, particularly for the complex scenarios discussed earlier. Your interviewer will be looking for you to be diving deep into at least 2-3 key areas, showcasing not just proficiency but also innovative thinking and optimal solution-finding abilities. They should show unique insights for at least a couple follow-up questions of increasing difficulty. A crucial indicator of a staff+ candidate's caliber is the level of insight and knowledge they bring to the table. A good measure for this is if the interviewer comes away from the discussion having gained new understanding or perspectives.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned.

Start Quiz

Mark as read ☐

How would you rate the quality of this article?

1

2

3

4

5

6

7

8

9

10

PoorGreat

Add a comment...

B

I

</>

99

≡

≡

🔗

👁

Comment

☐

Anonymous

Posting as leetcode

🔍

Search comments

Sort By

Popular

▼

A

Aleksey Klintsevich

★ Top 1%

• 1 year ago

• edited 9 hours ago

I think it might be worth it to mention the Saga pattern, when you're ordering the items, as a possible solution

👍

48

Reply

A

ApparentTomatoOx647 ★ Top 5% • 9 months ago • edited 7 days ago

+1

We can utilize Change Data Capture (CDC) or Outbox pattern on the Order table using pglogical, allowing us to track changes in order status (Ordered, Confirmed, or Failed). By adding the Order table to a replication set via `pglogical.replication_set_add_table`, we can capture and process these updates in near real time.

When a new order is placed, it is initially recorded with the status "Ordered". These changes can be then streamed by Postgres trigger + separate Kafka producer to a Kafka message broker, where a consumer group of worker processes listens for updates. Each worker independently processes incoming order events and attempts to update the Inventory table, ensuring consistency through row-level locking (fine-grained locking).

If a worker successfully deducts the required quantities from inventory, the order status is updated to "Confirmed". If the inventory update fails due to insufficient stock, the order is marked as "Failed" (reconciliation step), and the customer is notified.

Although customers must wait briefly for their order status to be confirmed, this event-driven architecture greatly enhances scalability by decoupling the Order and Inventory systems. This means that both databases can reside on separate hosts, reducing contention and improving overall system performance while it increases the system complexity.

[Show More](#) 23 [Reply](#)**slrparser** ★ Top 1% • 6 months ago • edited 2 days ago

It feels very overengineered to me, tbh.

 35 [Reply](#)**udit agrawal** ★ Top 10% • 5 months ago

With the intention to partition inventory table by `region_id`, it makes sense to keep order table in a separate database maybe sharded by `userid` (to fetch all the orders of a user), above event driven architecture makes much more sense.

 0 [Reply](#)**Max** Premium • 6 months ago • edited 1 month ago

I think it's very redundant to add CDC with Kafka, Kafka Connect to reduce transaction by this: `quantity = quantity - 1`.

 6 [Reply](#)**Nguyen Tran Trung** Premium • 21 days ago • edited 1 day ago

Yes agreed. The most important part is the compensation flow. You would Saga: 1) `ReserveOrder(itemId, dcId, amount)` on the Inventory table and 2) `INSERT INTO orders`.

If the first step fails then great. If second step fails, in case the service is alive, we can `RPC ReturnOrder(itemId, dcId, amount)` (this API must be idempotent, we will discuss right later). Harder stuff is when the Saga coordinator (Order Service) crashes during `INSERT` or during compensation, then we never call `ReturnOrder`. A good approach here is we can run scheduler service periodically to compensate failed orders. Since we failed the `INSERT`, we never knew which failed, so let's refactor the order flow by 1) `INSERT INTO orders (state, expires) VALUES (InProgress, 1 hours)`; 2) the second Saga step be `UPDATE orders SET state = Success`. Then we can always know which order is expired and stuck in `InProgress` to compensate (`CREATE INDEX idx_orders_state (expires, state)` and run this query in read-only slave to speed things up).

To make `ReturnOrder` idempotent - simply have a "compensated_reservation" table (`orderId, amount`) and check if the compensation is completed before adding amount to the items table during a transaction. This table can be vacuumed periodically during off hours (it's very small!)

This design allows decoupling orders and items/inventory table and shard them freely.

This is what I could come up with. any suggestions/simplification?

Show More

👍 1 Reply

N

NecessaryMoccasinEchidna420 ★ Top 5% • 1 year ago • edited 7 days ago

The whole inventory reservation thing feels more like an engineers' solution than a business goals solution.

Of course if the interviewer says you must *never* ever let someone place an order if you aren't 100% sure that you have stock, that's fine. Do it.

But if you're at a senior+ level and expected to show some business thinking beyond technical implementation and come up with that as one of the most important above-the-line requirements on your own... Is it really? What business goal are we fulfilling?

The business goal is to get people to spend money.

If you block them from checking out with 20 other items because a bag of chips in their cart went out of stock before they hit the "Place Order" button... If a 100ms latency increase on Amazon triggers a 1% drop in sales, what's the drop going to be for refusing to accept the order? The business will see the analytics funnel and tell you to remove this feature immediately.

The real-world business solution on these apps for items which *might* be out stock, but we don't know because of concurrency and also just-in-time supply chains is to give the customer a choice of substitution (preferable!) or refund for that one item.

Show More

👍 32 Reply

N

NecessaryMoccasinEchidna420 ★ Top 5% • 1 year ago • edited 7 days ago

Good Solution: Use Redis to "lock" items

Great Solution: Use a new status flag

Isn't this basically identical to the Ticketmaster ticket inventory solution, where the DB-based status lock with cron-process to release locks was considered sub-optimal and the Redis-based lock with TTL was considered "great"?

One last comment: this solution looks a lot more like a hotel reservation system than a grocery app, to be honest.

Although in real-life hotels and airlines allow overbooking, as business-wise to be "optimistic" and sell as much inventory as possible and disappoint some customers.

👍 6 Reply

C

ChemicalCopperIguana997 ★ Top 10% • 7 months ago • edited 7 days ago

In grocery apps, most SKUs are mass-quantity items, so the chance of inventory going to zero between "add to cart" and "checkout" is low. Retailers accept the small trade-off of occasional stock-outs at checkout in exchange for system simplicity and performance. Fulfillment can often be adjusted using substitution logic (e.g., offering a different brand of milk if one is unavailable), which keeps the customer experience smooth.

In contrast, systems like Ticketmaster or hotel booking platforms deal with scarce, uniquely identifiable resources (e.g., a specific concert seat or hotel room). In these cases, losing a booking due to concurrency issues is unacceptable to users. That's why these systems require strict lock management, race condition prevention, and mechanisms like Redis-based locks with TTL or DB row locking to ensure fairness and consistency.

👍 15 Reply



AliveJadeAntelope452 Premium • 11 months ago • edited 11 months ago

I was going to say the same thing about the similarities between this and hotel booking systems. Based on my understanding, they are roughly 90% the same, with only a few key differences:

1. Date Range: A hotel system query will always include a date range. Ensuring inventory item availability within a date range requires adjustments to the design. The inventory table will differ. Instead of handling multiple items per order, we primarily deal with one item per order, but with an associated date range. A significant challenge is maintaining

consistent room availability throughout the booked dates (I.e., preventing users from changing rooms during their stay).

2. Location/nearby service requirements is different. Hotel booking systems typically involve searching for remote locations, often by location name, rather than using the user's current lat/long coordinates. Determining the correct location presents a unique challenge. However, we don't need to consider how current traffic impacts commute time.

3. Traffic and Concurrency: Hotel booking systems often exhibit much higher spike ratios. Traffic can surge significantly near vacation seasons. High concurrency is also common for "hot items" (certain hotels on specific dates). This may necessitate infrastructure changes to handle the increased demand, such as implementing booking queues.

👍 2 [Reply](#)



Neethi Elizabeth Joseph • 1 year ago • edited 1 month ago

I wanted to ask about the same thing: TicketMaster vs GoPuff

The Distributed Lock is not considered great here.

I think leveraging the powers of the RDBMS is being prioritized here and we are leaning further into using RDBMS. I hope interviewers accept it.

👍 1 [Reply](#)



Kevin Blast • 1 year ago • edited 1 month ago

I think the difference is the ticketMaster locks down an unique resource (a ticket with unique seat) vs the inventory is a shared resource. Locking the inventory item isn't appropriate.

👍 2 [Reply](#)



Abhishek Agrawal • 1 year ago • edited 1 month ago

but we could be locking inventoryInstance. so yes, would be great to understand why Redis based lock with TTL is not great here.

👍 1 [Reply](#)



InstitutionalPurpleMollusk746 • 1 year ago • edited 1 month ago

I have same question here.. why is redis lock solution not considered a great solution here? Hope Stefan sees these comments and reply the reasoning.

👍 1 [Reply](#)



InstitutionalPurpleMollusk746 • 1 year ago • edited 21 days ago

Unlike Ticketmaster, when a user places an item for reservation in this application, they are more likely to complete the order rather than abandon the cart, since grocery items are typically needed sooner and the amount of money involved is smaller compared to ticket purchases. Therefore, periodic jobs to clean up the database for items with a 'reserved' status, allowing for some lag, are not a significant issue.

Additionally, unlike the singular quantity of a specific seat in Ticketmaster, this application usually deals with multiple quantities of item instances. As a result, if the count of item instances isn't perfectly accurate (e.g., 9 instead of 10), it won't matter as much to the user as long as there is more than 1 in stock. In contrast, for Ticketmaster, it's a binary situation where the seat is either available (1) or not (0).

Considering these trade-offs and the cost of maintaining Redis, I believe the second approach is better in this case.

Hope my reasoning makes sense!

👍 3 [Reply](#)



Tommy Loalbo • 11 months ago

I see your point about multiple in the inventory but potentially seeing something is out of or on hold by someone else or just lower in stock because of that could increase the scarcity effect and potentially drive up sales. I feel like it depends on the analytics.

👍 0 [Reply](#)

E

EasyCopperOtter704 Premium • 8 months ago

I had the same question. Maybe it is because inventory is usually not low (near zero) most of the time? You do not want double bookings, but it does not make sense to reserve items as your inventory is not low. Contrast that to the Ticketmaster problem where each event has only a few hundred tickets, and many people tend to book the same seats that they consider "best"

👍 0 [Reply](#)



Stefan Mai Admin • 1 year ago

Ha, this is a fair point. Most interviews are going to be synthetic in nature, you only have 1 hour after all. But if you were designing this in the real world you're right, you'd probably want to design some partial checkout functionality.

👍 2 [Reply](#)

N

NecessaryMoccasinEchidna420 ★ Top 5% • 1 year ago

Yep, that makes sense. Tbh, if I were an interviewer and someone used this prompt as an opportunity to show how they would design a real-time inventory system, it's probably fine.

But as someone who has a Meta Interview coming up in a couple of weeks for E6, this blog post makes me mildly anxious wondering if an interviewer who asks this particular question would ding me because I only designed a "best effort" stock system and prioritized other goals

👍 0 [Reply](#)



Stefan Mai Admin • 1 year ago

System design questions don't have 1 right answer: the important piece is you being able to demonstrate that you can think on your feet, use your body of experience to make tradeoffs, and creatively build a solution.

This question isn't appropriate for an E6 interview, FWIW. Keep in mind that for E6 you'll want to go deeper than most of the guides on this site cover.

👍 1 [Reply](#)



Stefan Mai Admin • 1 year ago

We'll have to work on that next. Much smaller audience!

👍 4 [Reply](#)

N

NecessaryMoccasinEchidna420 ★ Top 5% • 1 year ago • edited 7 days ago

True, but then most prep material is focused at a lower level, it's very hard to find good examples of staff-level system design that's also digestible as a refresher within a limited time window! I haven't interviewed in 5 years at this point. I read through DDIA and some of its references to get a bit more depth. Still feel pretty unprepared though.

👍 1 [Reply](#)

L

LooseCopperConstrictor421 • 1 year ago

I think best effort solution makes sense here because of the delivery constraint of 1 hour. Had it been like amazon where it's a matter of couple of days or a week in some cases, overbooking makes sense.

👍 0 [Reply](#)

N

NecessaryMoccasinEchidna420 ★ Top 5% • 1 year ago

Also am I reading the design correctly that we have a ItemInstance row in the database for each individual item? If we have 1 million packs of gum in inventory we have 1 million rows?

👍 0 [Reply](#)

D **Dota Warning** • 11 months ago • edited 1 month ago

If one row signifies one item, why do we have quantity in inventory schema?

👍 4 [Reply](#)

 **Stefan Mai** Admin • 1 year ago

Yes.

👍 0 [Reply](#)

E **EquivalentPurpleMarmoset761** Premium • 1 year ago

Why not using a quantity field for each item instance?

👍 1 [Reply](#)

S **Sheraz Khan** • 1 year ago

If I understand your proposal correctly, you are saying to have a table say "Reservation" which will hold a row per item that is added to cart. While the item is added in this reservation table, inventory is reduced by same quantity. Once the order is confirmed, we remove the row from reservation table and insert in Order table. If order is cancelled, we remove row reservation table and update inventory table. That's what I first thought when I read the article.

👍 0 [Reply](#)

W **Wang lei** Premium • 1 year ago

It is not scalable then, if they are not sold, they are waste of space then just with a quantity. Per my experience, we can delay its creation when it is purchased.

👍 0 [Reply](#)

E **ExtraAmaranthDuck389** ★ Top 5% • 7 months ago • edited 1 day ago

This feels a bit too high level.

1. Should all items be available/ordered from a single DC? Once the neabyService provides a list of DC_ID, are we checking if all items are available in any single DC or across DCs?
2. For available, how do we cache in Redis?
3. How are we syncing DC data from DB to our application server in-memory cache?
4. Partition: IIUC, the DC_ID will contain a prefix of Zip Code. And we partition our table on that.

Let's say there is a single application server which can server 20K search qps. But DB can only do like 5K. So then we will have 4 partitions based on zip code. Then the single application server will call the DB and the DB has the knowledge of which of the 4 partition to call?

1. Replicas: Can you go in more detail how replication works in this case? How many replicas to use? How long does it take for the replicas to be updated with the master? How do we check if they are actually getting updated?

👍 16 [Reply](#)

R **RightBlueHippopotamus746** • 2 months ago

Should all items be available/ordered from a single DC? Once the neabyService provides a list of DC_ID, are we checking if all items are available in any single DC or across DCs?

Seems like a requirements clarification question, which you should align with your interviewer based on the problem focus.

For available, how do we cache in Redis?

The article has been updated with the flow:

-Check cache for availability.

- If unavailable, check DB and write to cache.
- Cache items with a TTL of 1 min to keep them fresh.
- Revoke cache items when Order updates inventory values.

How are we syncing DC data from DB to our application server in-memory cache?

The app servers can periodically read the values from the DB, and update the In memory cache.

Let's say there is a single application server which can server 20K search qps. But DB can only do like 5K. So then we will have 4 partitions based on zip code. Then the single application server will call the DB and the DB has the knowledge of which of the 4 partition to call?

[Show More](#)

👍 0 [Reply](#)

A **akakhilkumbar** Premium • 7 months ago • edited 1 month ago

need a back to top button, Please

👍 10 [Reply](#)

G **GothicCopperPanther728** Premium • 3 months ago • edited 7 days ago

Command + up arrow

👍 5 [Reply](#)

A **Adil Hussain** Premium • 7 days ago

For mobile readers this won't work.

👍 0 [Reply](#)

C **CivicBlackAsp163** • 1 year ago

Why not use a queue like Kafka or SQS? The order processing can be async, and it will just have more logic as time goes on.

👍 7 [Reply](#)

[Show All Comments](#)

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

[Schedule A Mock Interview](#)

Questions

[Meta SWE Interview Questions](#)

[Amazon SWE Interview Questions](#)

[Google SWE Interview Questions](#)

[OpenAI SWE Interview Questions](#)

[Engineering Manager \(EM\) Interview Questions](#)

Learn

[Learn System Design](#)

[Learn DSA](#)

[Learn Behavioral](#)

[Learn ML System Design](#)

[Learn Low Level Design](#)

[Guided Practice](#)

Links

[FAQ](#)

[Pricing](#)

[Gift Mock Interviews](#)

[Gift Premium](#)

[Become a Coach](#)

[Our Coaches](#)

[Hello Interview Premium](#)

Legal

[Terms and Conditions](#)

[Privacy Policy](#)

Contact

[About Us](#)

[Product Support](#)

7511 Greenwood Ave North

Unit #4238 Seattle

WA 98103